

Operations and Complexity

Function bubble_sort

Number of operations is $n*(n-1) / 2$, because when it comes to comparison in the worst case, the inner loop becomes shorter after every pass. Then we will get:

$$(n-1) + (n-2) + \dots + 1$$

By using arithmetic-series formula:

$$(n-1)*n / 2$$

The Big-O complexity is $O(n^2)$ because the dominant term in $(n-1)*n / 2$ is n^2 .

Function user_score

Number of operations is constant because the function only contains a fixed number of if statement and arithmetic operations. It doesn't have any loops, and the amount of array do not depend on the size of array.

The Big-O complexity is $O(1)$ because the same fixed number of operations is performed regardless of the input size.

Function linear_search

Number of operations is n in the worst case because the target may either be the last element in the array do not exist in the array.

The Big-O complexity is $O(n)$ because the number of operations grows linearly with the size of array.

Function count_liked

Number of operations is $l*(c + s)$ because the loop runs l times and for each loop it performs two linear searches. The number of operations is approximately $l*(c + s)$.

The Big-O complexity is $O(n^2)$ because when l , c and s are approaching infinite, then $O(l*(c + s)) = O(2n^2) = O(n^2)$

Function binary_search

Number of operations is $\log_2 n$ because each iteration removes half of the remaining array.

The Big-O complexity is $O(\log n)$ because the number of operations is $\log_2 n$. The search space is divided by 2 during each iteration.

Function count_comment_no_like

Number of operations is $c \cdot (\log_2 l + s)$ because the outer loop runs c times, and for every comment would be executed by binary search and if user doesn't like the post, it would be executed by linear search as well.

The Big-O complexity is $O(n \log n)$ because when l , c and s are approaching infinite, then $O(c \cdot (\log_2 l + s)) = O(n \log n)$

Function count_only_share

Number of operations is $s \cdot (\log_2 l + c)$ because the outer loop runs s times, and for every share would be executed by binary search and linear search.

The Big-O complexity is $O(n \log n)$ because when l , c and s are approaching infinite, then $O(s \cdot (\log_2 l + c)) = O(n \log n)$

Function engagement_score

Number of operations is $(l-1) \cdot l / 2 + l \cdot (c + s) + c \cdot (\log_2 l + s) + s \cdot (\log_2 l + c)$ because the function firstly calls bubble_sort. It then calls count_liked, count_comment_no_liked and count_only_share.

The Big-O complexity is $O(n^2)$ because if l , c and s grow proportionally with the input size n , the dominant term are quadratic terms. Lower-order terms like $n \log n$ could be ignored in Big-O notation.

Execution time was measured on input sizes 100, 500, 1000, 2000, 4000 and the results were 615 ms, 13556 ms, 45177 ms, 93090 ms, 370084 ms.

Evidence of Measurement

```

void test_execution_time(int n) {
    string* likes = new string[n];
    string* comments = new string[n];
    string* shares = new string[n];

    for (int i = 0; i < n; i++) {
        likes[i] = "user" + to_string(i);
        comments[i] = "user" + to_string(i);
        shares[i] = "user" + to_string(i);
    }

    auto start :time_point<steady_clock, steady_clock::duration> = chrono::high_resolution_clock::now();

    int score = engagement_score(
        likes, l: n,
        comments, c: n,
        shares, s: n
    );

    auto end :time_point<steady_clock, steady_clock::duration> = chrono::high_resolution_clock::now();

    auto duration =
        chrono::duration_cast<chrono::microseconds>(fd: end - start);

    cout << "Input size n = " << n
        << ", Execution time = "
        << duration.count()
        << " microseconds"
        << endl;

    delete[] likes;
    delete[] comments;
    delete[] shares;
}

```

```

int main() {
    /*
    string likes[4] = {"Nora", "Ethan", "Maya", "Liam"};
    string comments[3] = {"Ethan", "Maya", "Zara"};
    string shares[3] = {"Nora", "Maya", "Oscar"};
    int l = 4;
    int c = 3;
    int s = 3;
    int score = engagement_score(likes, l, comments, c, shares, s);
    cout << "Engagement score for this post is " << score << endl;
    return 0;
    */

    test_execution_time(n: 100);
    test_execution_time(n: 500);
    test_execution_time(n: 1000);
    test_execution_time(n: 2000);
    test_execution_time(n: 4000);
}

```

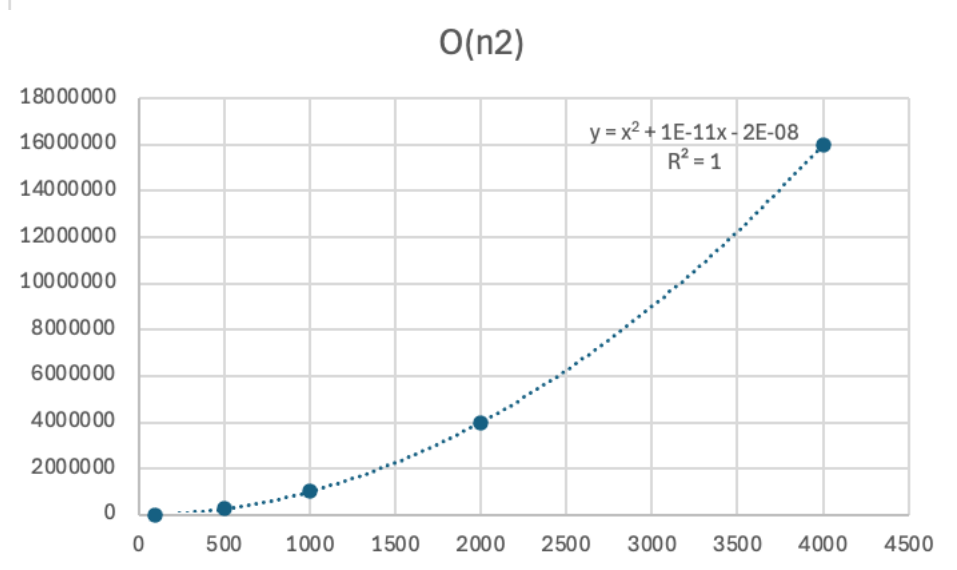
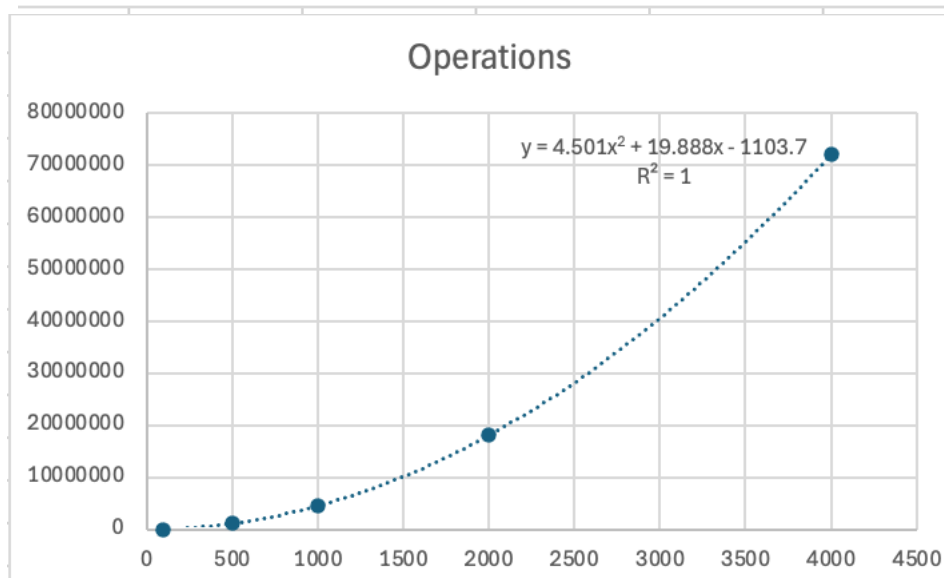
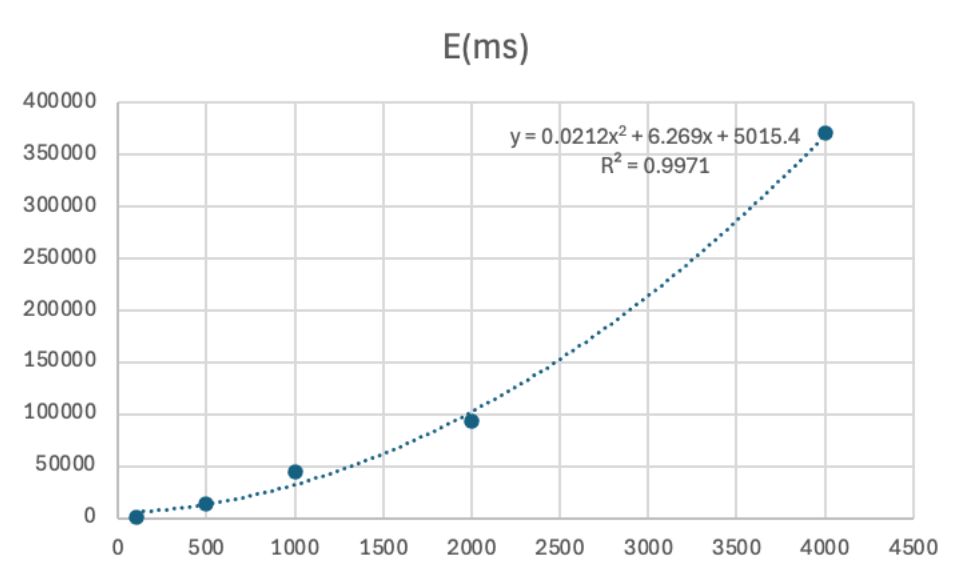
```

/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P4/P4_task
Input size n = 100, Execution time = 615 microseconds
Input size n = 500, Execution time = 13556 microseconds
Input size n = 1000, Execution time = 45177 microseconds
Input size n = 2000, Execution time = 93090 microseconds
Input size n = 4000, Execution time = 370084 microseconds

Process finished with exit code 0

```

Charts



Conclusion

If a different input were sorted instead of likes, the final big-O notation would still be $O(n^2)$. This is mainly because bubble sort itself has a complexity of $O(n^2)$, and the remaining functions still contain combination of loops and linear search produces quadratic behaviour. The actual execution time depends on the relative size of likes, comments and shares and how often each array is searched.

$$\begin{aligned}
 & (l-1) * l / 2 + l * (c + s) + c * (\log_2 l + s) + s * (\log_2 l + c) \\
 &= l_2/2 - l/2 + l * c + l * s + 2 * c * s + (c+s) * \log_2 l \\
 &= l_2/2 - l/2 + 2 * c * s + (c+s) * (\log_2 l + l)
 \end{aligned}$$

The graph shows the measured execution time literally follows the same increasing trend as the estimated number of operations and theoretical complexity. As the input increases, all three lines grow at a similar quadratic rate.

Word Count

This document contains 483 words.